
GRAFIKA KOMPUTEROWA LAB_8

Table of Contents

1. CEL CWICZENIA	1
2. ZADANIA	1
2.0 Funkcje pomocnicze	1
2.0.1 Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie z poprzednich laboratoriów	1
2.0.2 Funkcja pomocnicza z laboratorium 04 wypełniająca kontur	3
2.0.3 Funkcja do wyznaczania parametrów płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty $p1, p2, p3$	4
2.0.4 Funkcja do wyznaczania odległości punktu $[x, y, z]$ od płaszczyzny o równaniu $Ax+By+Cz+D=0$	5
2.1 ZADANIE 1	5
(a) $lampa = [-3; -3; -10];$	7
(b) $lampa = [3; 3; 3];$	7
2.2 ZADANIE 2	9
(a) $lampa = [-3; -3; -10];$	11
(b) $lampa = [3; 3; 3];$	11
2.3 ZADANIE 3	12
(a) model LAMBERTA z lampą a	12
(b) model PHONGA z lampą a	13
Typowy koniec skryptu	13

Joanna Dagil 231008

1. CEL CWICZENIA

Modele oświetlenia i cieniowania.

2. ZADANIA

2.0 Funkcje pomocnicze

```
clear; close all; clc;
```

2.0.1 Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie z poprzednich laboratoriów

```
function [i,j] = mat_to_pix(x, y, M, N, dx, dy)
    T = [1/dx, 0, M/2;
         0, -1/dy, N/2;
         0, 0, 1];
    temp = [x; y; 1];
    res = T * temp;
```

```
i = round(res(1)/res(3));
j = round(res(2)/res(3));
end

function [i,j] = pix_to_mat(x, y, M, N, dx, dy)
    T = [dx, 0, -M*dx/2;
         0, -dy, N*dy/2;
         0, 0, 1];
    res = T * [x; y; 1];
    i = res(1)/res(3);
    j = res(2)/res(3);
end

function Im2 = linia1(Im1,yp,xp,yk,xk,C)
    % na monochromatycznym obrazie kresli linie o jasnosci C
    % od punktu (yp,xp) do punktu (yk, xk)
    [Y, X] = size(Im1);
    Im2 = Im1;

    if (abs(yk-yp) > abs(xk-xp)) % stroma linia
        x0 = yp;y0 = xp; x1 = yk;y1 = xk; % zamiana wspolrzecznych
        zamiana = 1;
    else
        x0 = xp;y0 = yp; x1 = xk;y1 = yk;
        zamiana = 0;
    end
    if zamiana==1
        temp=X;
        X=Y;
        Y=temp;
    end % zamiana rozmiarow X i Y
    if(x0 >x1) % zamiana poczatku i konca linii, zeby zaczynac od lewej
        temp1 = x0; x0 = x1; x1 = temp1;
        temp2 = y0; y0 = y1; y1 = temp2;
    end
    dx = abs(x1 - x0) ; % odleglosc x
    dy = abs(y1 - y0); % odleglosc y
    sy = sign(y1 - y0); % znak przyrostu w kierunku y
    x = x0; y = y0; % inicjalizacja
    if x>0 && x<=X && y>0 && y<=Y
    if (zamiana ==0)
        Im2(y,x) = C; % rysowanie punktu
    else
        Im2(x,y) = C;
    end
    end
    error = 2*dy - dx ; % inicjalizacja bledu
    for i = 0:dx-1
        error = error + 2*dy; % modyfikacja bledu
        if (error >0)
            y = y +sy;
            error = error - 2*dx;
        end
        x = x + 1; % zwiekszamy x
    end
end
```

```
if x>0 && x<=X && y>0 && y<=Y
    if (zamiana ==0)
        Im2(y,x) = C;                % rysowanie punktu
    else
        Im2(x,y) = C;
    end
end
end
end
```

2.0.2 Funkcja pomocnicza z laboratorium 04 wypełniająca kontur

```
function Im2 = floodfill1(Im, y, x, T, C)
[H, W] = size(Im);

% sprawdzenie zakresu
if y < 1 || y > H || x < 1 || x > W
    error('Punkt startowy (y,x) jest poza obrazem.');
```

```
end

% sprawdzenie jasności piksela startowego
if Im(y,x) ~= T
    error('Piksel startowy (%d,%d) ma jasność %d zamiast T=%d.', ...
        y, x, Im(y,x), T);
end

% jeśli nowa jasność taka sama jak stara, nic nie trzeba robić
if T == C
    Im2 = Im;
    return;
end

Im2 = Im;

% kolejka pikseli do odwiedzenia
Q = zeros(H*W, 2);
head = 1;
tail = 1;

Q(tail,:) = [y, x];
Im2(y,x) = C;

while head <= tail
    cy = Q(head,1);
    cx = Q(head,2);
    head = head + 1;

    % 4-sąsiedztwo: góra, dół, lewo, prawo

    % góra
    ny = cy - 1;
```

```
nx = cx;
if ny >= 1 && Im2(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im2(ny,nx) = C;
end

% dol
ny = cy + 1;
nx = cx;
if ny <= H && Im2(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im2(ny,nx) = C;
end

% lewo
ny = cy;
nx = cx - 1;
if nx >= 1 && Im2(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im2(ny,nx) = C;
end

% prawo
ny = cy;
nx = cx + 1;
if nx <= W && Im2(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im2(ny,nx) = C;
end
end
end
```

2.0.3 Funkcja do wyznaczania parametrów płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty p1,p2,p3

```
function [A, B, C, D] = plane(p1,p2,p3)
A = det([p1(2),p1(3),1;
        p2(2),p2(3),1;
        p3(2),p3(3),1]);
B = -det([p1(1),p1(3),1;
        p2(1),p2(3),1;
        p3(1),p3(3),1]);
C = det([p1(1),p1(2),1;
        p2(1),p2(2),1;
        p3(1),p3(2),1]);
D = -det([p1(1),p1(2),p1(3);
```

```
        p2(1), p2(2), p2(3);  
        p3(1), p3(2), p3(3)]);  
end
```

2.0.4 Funkcja do wyznaczania odległości punktu $[x,y,z]$ od płaszczyzny o równaniu $Ax+By+Cz+D=0$

% odległość mierzona jest wzdłuż prostej o wektorze kierunkowym $[l,m,n]$
% zwraca również punkt przecięcia z płaszczyzną

```
function [d,x1,y1,z1] = dist2plane(A, B, C, D, x, y, z, l, m, n)  
  
    ro = (A*x+B*y+C*z+D)/(A*l+B*m+C*n);  
    x1 = x-l*ro; y1 = y-m*ro; z1 = z-n*ro;  
    d = sqrt((x-x1)^2+(y-y1)^2+(z-z1)^2);  
end
```

2.1 ZADANIE 1

% Według modelu LAMBERTA trójkąt o wierzchołkach

```
p1 = [9, 3, 16, 1];  
p2 = [-3, 9, 6, 1];  
p3 = [-3, -3, 4, 1];
```

% oświetlony światłem rozproszonym o natężeniu 20
ambient = 20;

% oraz punktowym źródłem światła o natężeniu 235
int = 235;

% umieszczonym w punkcie

% (a)

```
lampa = [-3; -3; -10];
```

% (b)

```
lampb = [3; 3; 3];
```

% współczynnik odbicia

```
e = 1;
```

% współczynnik (określający jak szybko natężenie oświetlenia maleje wraz z odległością od źródła)

```
alpha = 0.001;
```

% rozdzielczość

```
X = 256;
```

```
Y = 256;
```

% rozmiar pixela

```
dx = 0.1;
```

```
dy = 0.1;

Im = uint8(zeros(Y, X)); % Inicjalizacja obrazu

function Im1 = tr_Lambert(Im, p1, p2, p3, ambient, lamp, int, X, Y, dx, dy,
alpha, e)
    % Obliczanie wektora normalnego do płaszczyzny trójkąta
    Im1 = Im; % Inicjalizacja obrazu
    Ish = uint8(zeros(Y,X)); % pomocniczy obraz zawierający biały trójkąt

    [i1,j1] = mat_to_pix(p1(1), p1(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(p2(1), p2(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(p3(1), p3(2), X, Y, dx, dy);

    Ish = linia1(Ish, j1, i1, j2, i2, 255);
    Ish = linia1(Ish, j2, i2, j3, i3, 255);
    Ish = linia1(Ish, j3, i3, j1, i1, 255);

    Ish = floodfill1(Ish, round((j1+j2+j3)/3), round((i1+i2+i3)/3), 0, 255);
    % wypełnienie trójkąta białym kolorem

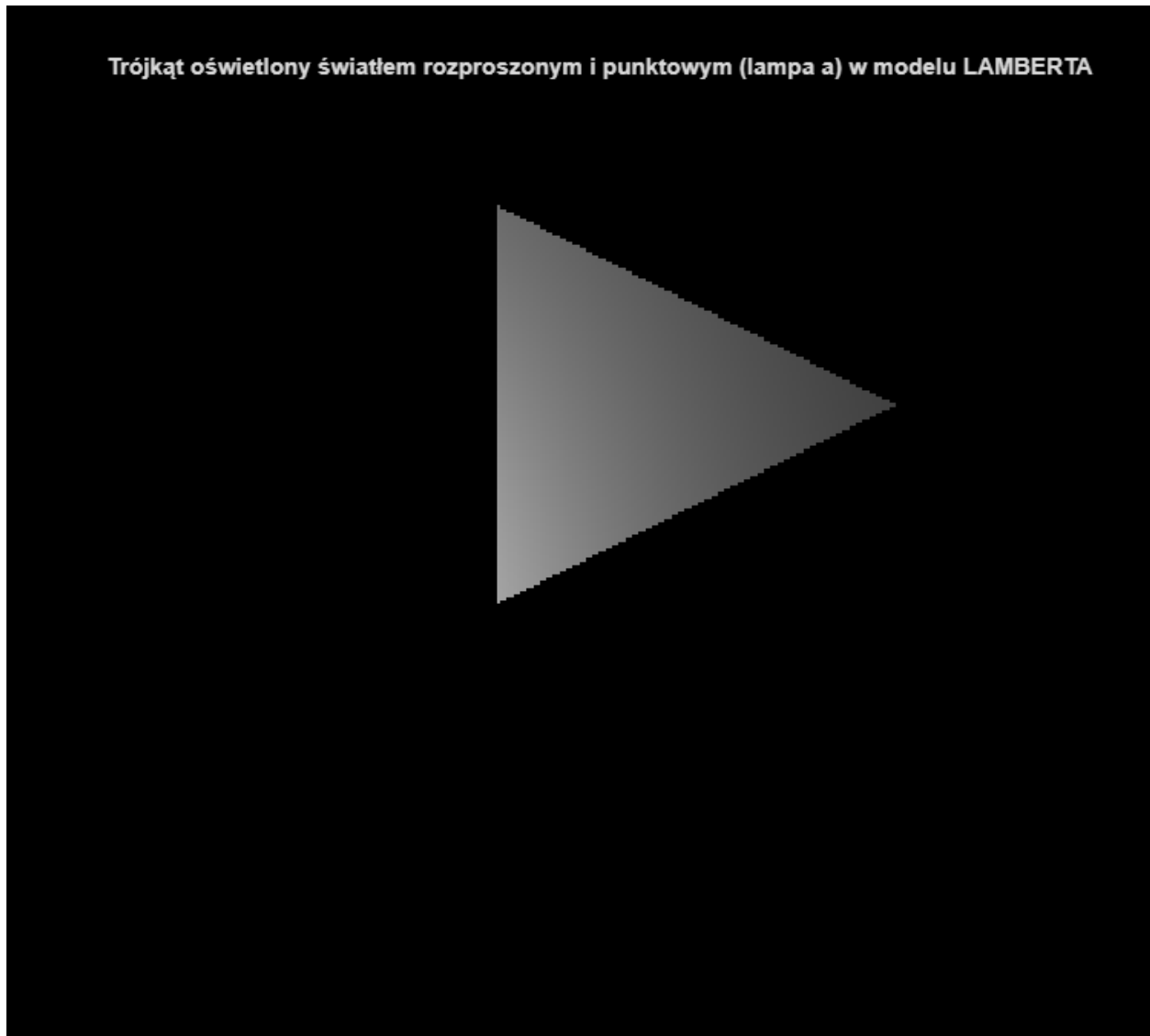
    [A, B, C, D] = plane(p1, p2, p3); % Wyznaczenie parametrów płaszczyzny
    trójkąta
    n = [A; B; C] / norm([A; B; C]); % Normalizacja wektora normalnego
    if n(3) > 0
        n = -1*n;
    end % Obrócenie normalnej, jeśli jest skierowana w dół

    for i = 1:X
        for j = 1:Y
            if Ish(j,i) == 255 % Sprawdzenie, czy piksel jest wewnątrz
                trójkąta
                    [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy); % Zamiana
                    współrzędnych pikselowych na matematyczne
                    [~, x, y, z] = dist2plane(A, B, C, D, xx, yy, 0, 0, 0, 1); %
                    Wyznaczenie punktu na płaszczyźnie trójkąta
                    l = lamp - [x; y; z]; % Wektor od punktu na trójkącie do lampy
                    dist = norm(l); % Odległość od lampy
                    l = l / dist; % Normalizacja wektora l
                    cs = n'* l;
                    if cs < 0
                        cs = 0;
                    end % Obliczenie cosinusa kąta między normalną a wektorem do
                    lampy
                    temp = ambient + int * cs * e / (1 + alpha * dist^2); %
                    Obliczenie natężenia oświetlenia z uwzględnieniem tłumienia
                    if temp > 255
                        temp = 255;
                    end % Ograniczenie natężenia do 255
                    Im1(j, i) = temp; % Ustawienie natężenia dla pikseli wewnątrz
                    trójkąta
                end
            end
        end
    end
```

```
end  
end
```

(a) lampa = [-3; -3; -10];

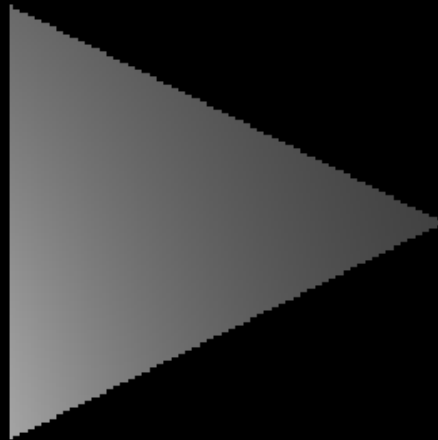
```
Im = tr_Lambert(Im, p1, p2, p3, ambient, lampa, int, X, Y, dx, dy, alpha, e);  
figure(11); imshow(Im); title('Trójkąt oświetlony światłem rozproszonym i  
punktowym (lampa a) w modelu LAMBERTA');
```

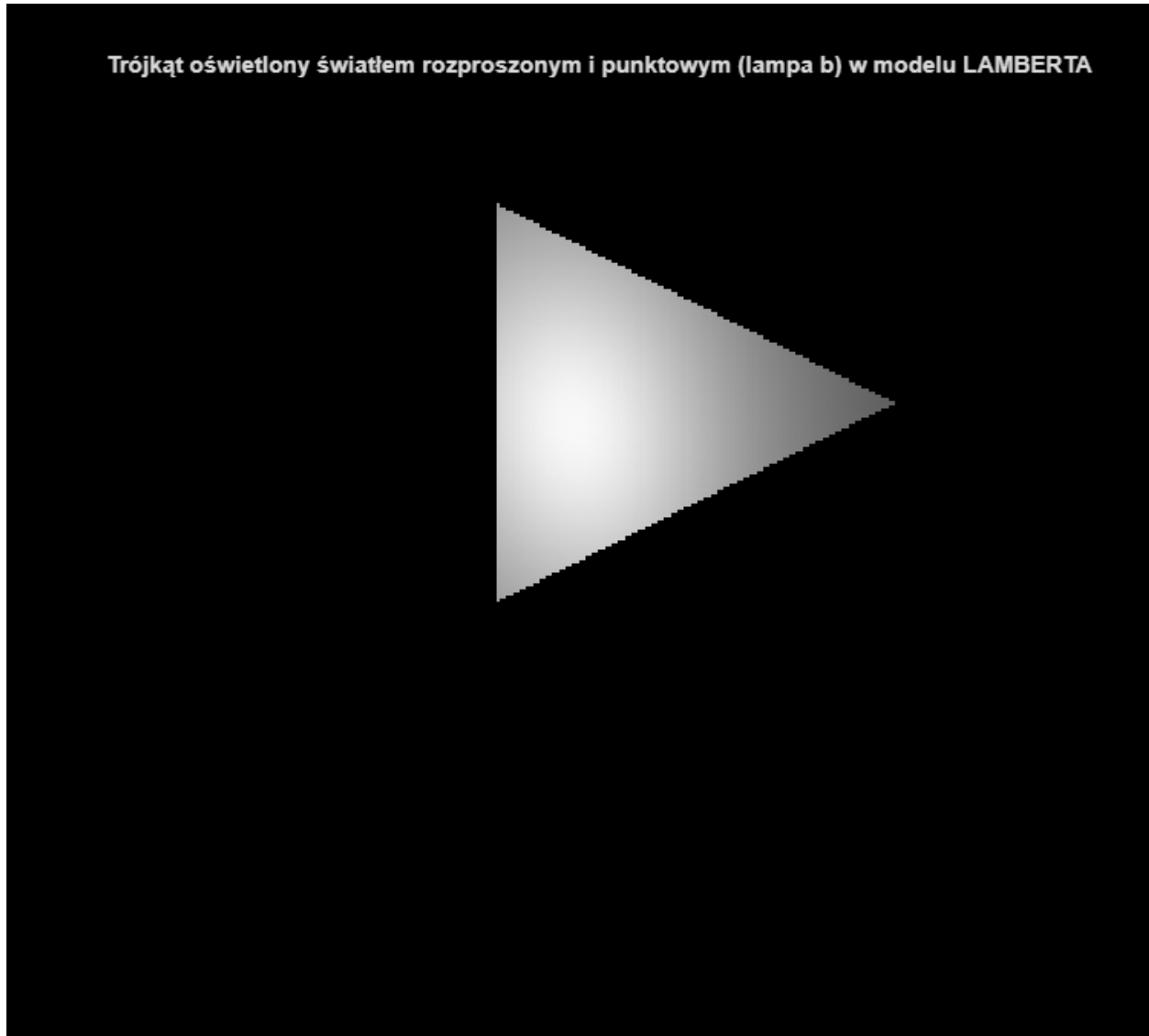


(b) lampb = [3; 3; 3];

```
Im = tr_Lambert(Im, p1, p2, p3, ambient, lampb, int, X, Y, dx, dy, alpha, e);  
figure(12); imshow(Im); title('Trójkąt oświetlony światłem rozproszonym i  
punktowym (lampa b) w modelu LAMBERTA');
```

Trójkąt oświetlony światłem rozproszonym i punktowym (lampa a) w modelu LAMBERTA





2.2 ZADANIE 2

```
% j.w. według modelu PHONGA z okiem obserwatora ustawionym w punkcie  
eye = [5; 5; 3];
```

```
% dla różnych potęg cosinusa  
m = 10;
```

```
function Im1 = tr_Phong(Im, p1, p2, p3, ambient, lamp, int, X, Y, dx, dy,  
alpha, e, eye, m)  
    % Obliczanie wektora normalnego do płaszczyzny trójkąta  
    Im1 = Im; % Inicjalizacja obrazu  
    Ish = uint8(zeros(Y,X)); % pomocniczy obraz zawierający biały trójkąt  
  
    [i1,j1] = mat_to_pix(p1(1), p1(2), X, Y, dx, dy);  
    [i2,j2] = mat_to_pix(p2(1), p2(2), X, Y, dx, dy);
```

```

[i3,j3] = mat_to_pix(p3(1), p3(2), X, Y, dx, dy);

Ish = linia1(Ish, j1, i1, j2, i2, 255);
Ish = linia1(Ish, j2, i2, j3, i3, 255);
Ish = linia1(Ish, j3, i3, j1, i1, 255);

Ish = floodfill1(Ish, round((j1+j2+j3)/3), round((i1+i2+i3)/3), 0, 255);
% wypełnienie trójkąta białym kolorem

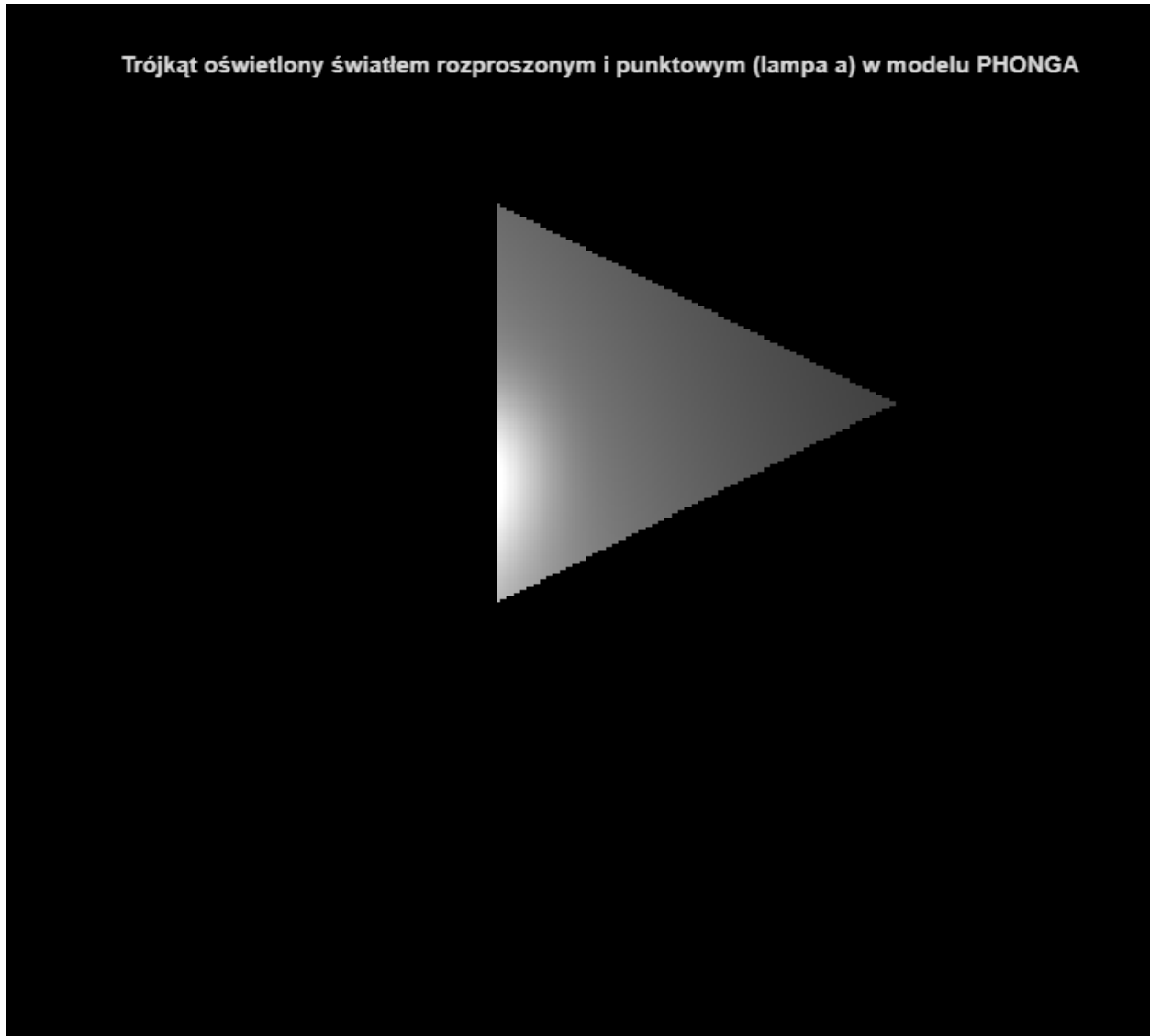
[A, B, C, D] = plane(p1, p2, p3); % Wyznaczenie parametrów płaszczyzny
trójkąta
n = [A; B; C] / norm([A; B; C]); % Normalizacja wektora normalnego
if n(3) > 0
    n = -1 * n;
end % Obrócenie normalnej, jeśli jest skierowana w dół

for i = 1:X
    for j = 1:Y
        if Ish(j,i) == 255 % Sprawdzenie, czy piksel jest wewnątrz
trójkąta
            [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy); % Zamiana
współrzędnych pikselowych na matematyczne
            [~, x, y, z] = dist2plane(A, B, C, D, xx, yy, 0, 0, 0, 1); %
Wyznaczenie punktu na płaszczyźnie trójkąta
            l = lamp - [x; y; z]; % Wektor od punktu na trójkącie do lampy
            dist = norm(l); % Odległość od lampy
            l = l / dist; % Normalizacja wektora l
            cs = n' * l;
            if cs < 0
                cs = 0;
            end % Obliczenie cosinusa kąta między normalną a wektorem do
lampy
            rp = 2 * (n' * l) * n - l; % Wektor odbicia
            rp = rp / norm(rp); % Normalizacja wektora odbicia
            vp = eye - [x; y; z]; % Wektor od punktu na trójkącie do oka
            vp = vp / norm(vp); % Normalizacja wektora do oka
            cs2 = rp' * vp;
            if cs2 < 0
                cs2 = 0;
            end % Obliczenie cosinusa kąta między wektorem odbicia a
wektorem do oka
            temp = ambient + int * cs * e * (1 + (cs2^m)) / (1 + alpha *
dist^2); % Obliczenie natężenia oświetlenia z uwzględnieniem tłumienia
            if temp > 255
                temp = 255;
            end % Ograniczenie natężenia do 255
            Im1(j, i) = temp; % Ustawienie natężenia dla pikseli wewnątrz
trójkąta
        end
    end
end
end
end

```

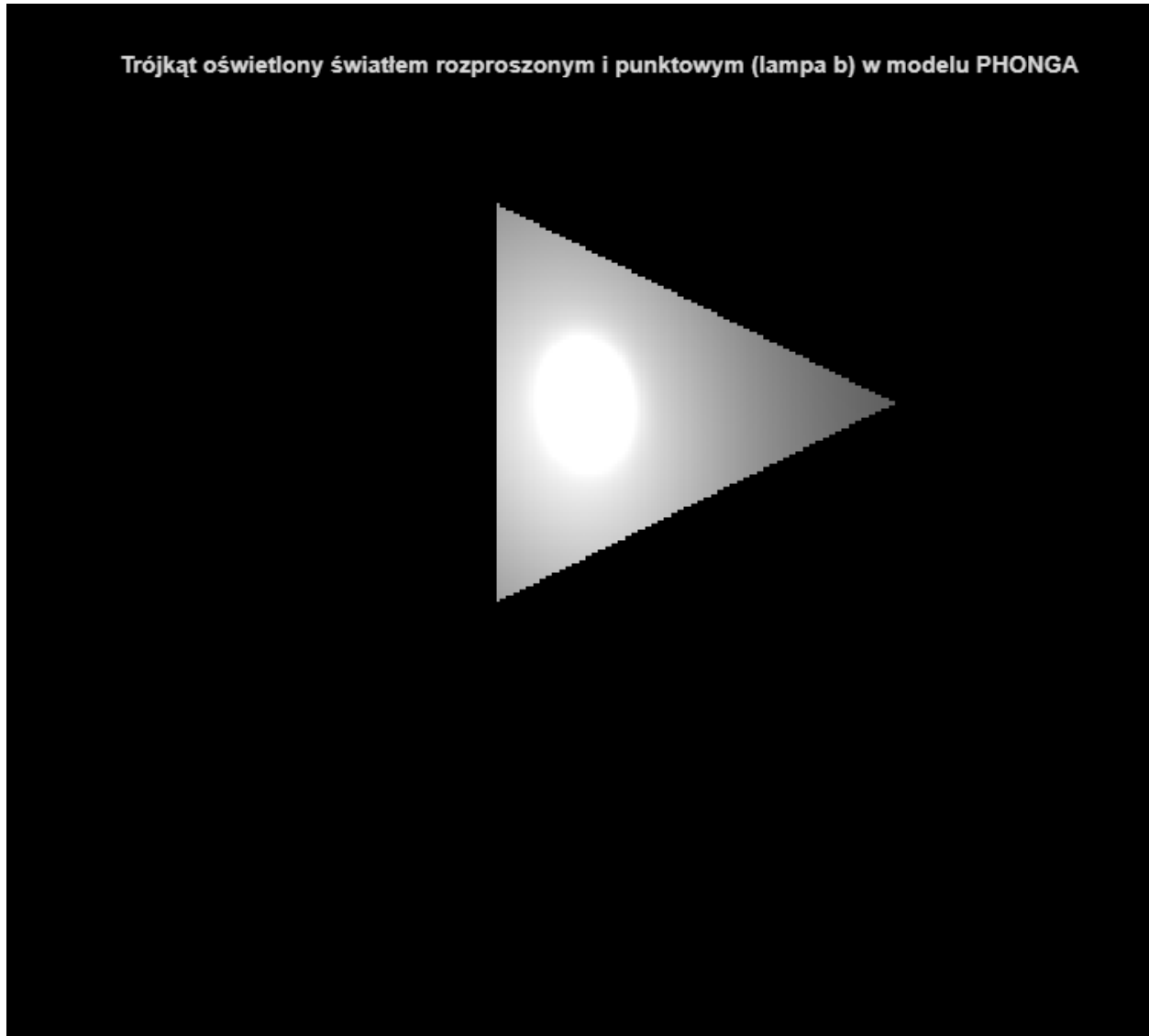
(a) $\text{lampa} = [-3; -3; -10];$

```
Im = tr_Phong(Im, p1, p2, p3, ambient, lampa, int, X, Y, dx, dy, alpha, e,  
eye, m);  
figure(21); imshow(Im); title('Trójkąt oświetlony światłem rozproszonym i  
punktowym (lampa a) w modelu PHONGA');
```



(b) $\text{lampb} = [3; 3; 3];$

```
Im = tr_Phong(Im, p1, p2, p3, ambient, lampb, int, X, Y, dx, dy, alpha, e,  
eye, m);  
figure(22); imshow(Im); title('Trójkąt oświetlony światłem rozproszonym i  
punktowym (lampa b) w modelu PHONGA');
```



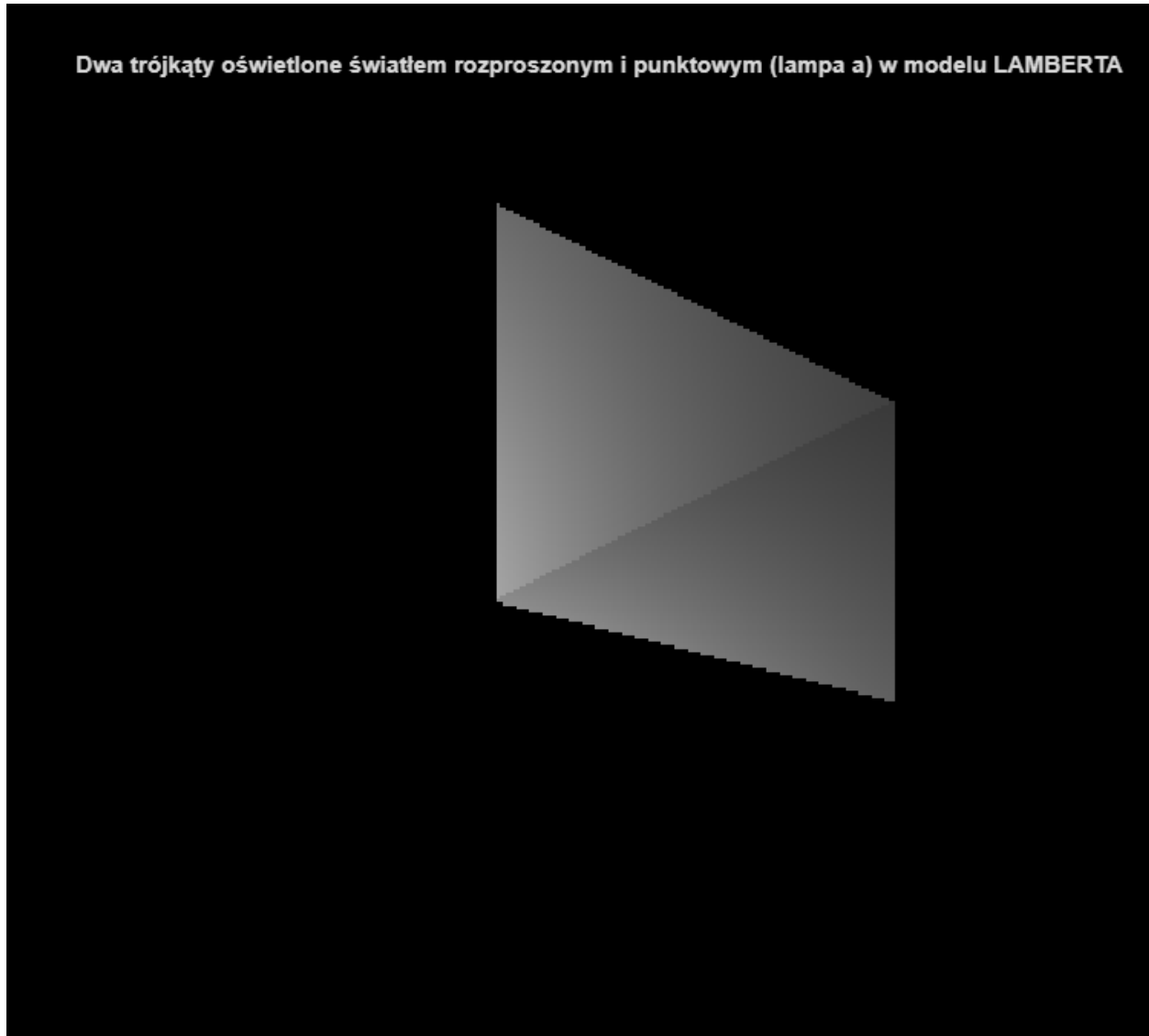
2.3 ZADANIE 3

% j.w. dla 2 trójkątów przylegających do siebie o wierzchołkach p1, p2, p3 oraz p1, p3, p4, gdzie

```
p1 = [ 9,  3, 16, 1];  
p2 = [-3,  9,  6, 1];  
p3 = [-3, -3,  4, 1];  
p4 = [ 9, -6,  5, 1];
```

(a) model LAMBERTA z lampą a

```
Im = tr_Lambert(Im, p1, p2, p3, ambient, lampa, int, X, Y, dx, dy, alpha, e);  
Im = tr_Lambert(Im, p1, p3, p4, ambient, lampa, int, X, Y, dx, dy, alpha, e);  
figure(31); imshow(Im); title('Dwa trójkąty oświetlone światłem rozproszonym  
i punktowym (lampa a) w modelu LAMBERTA');
```



(b) model PHONGA z lampą a

```
Im = tr_Phong(Im, p1, p2, p3, ambient, lampa, int, X, Y, dx, dy, alpha, e,  
eye, m);  
Im = tr_Phong(Im, p1, p3, p4, ambient, lampa, int, X, Y, dx, dy, alpha, e,  
eye, m);  
figure(32); imshow(Im); title('Dwa trójkąty oświetlone światłem rozproszonym  
i punktowym (lampa a) w modelu PHONGA');
```

Typowy koniec skryptu

```
% publish('lab08.m', 'pdf')  
  
% clear all; % usuniecie wszystkich zmiennych
```

```
% close all; % zamkniecie wszystkich wykresow  
% clc % zresetowanie okna komend
```

Published with MATLAB® R2025b